

# The QueryGraph Stack

A governed semantic lakehouse for agentic AI — five coordinated open-source components that let agents answer over enterprise data while *proving* what they did.

## THE THESIS

“The answer used OSI metric **X**, resolved to Sail table **Y**, under capability **C** and `odr1:read`, emitting OpenLineage run **R** anchored by attestation **A** — and source **Z** was denied, with a receipt.” The differentiator for enterprise AI is not a longer context window; it is this verifiable chain from question to answer. Denials are receipts, never errors.

## THE COMPONENTS

| Component   | Role                                                                                                                                                                                      | Release           |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| Grust       | Backend-neutral property graph for Rust; full GQL/Cypher reads (Full39075: CALL subqueries, TVFs, shortestPath); a dozen stores including Sail DataFrames.                                | 0.12.0 “Lobster”  |
| TypeSec     | Security in the type system: unforgeable Capability<P, R> proofs; TypeDID Ed25519 signed agent envelopes; framework guards, MCP gate, enforcement proxy.                                  | 0.12.0 “Torcello” |
| LakeCat     | Rust-native Iceberg REST catalog with stock-client conformance; catalog state, Sail planning, receipts, and graph projection bound to one table transition; QueryGraph bootstrap bundles. | 0.3.0 “Ocelot”    |
| Sail (fork) | Spark-compatible lakehouse engine with a Cypher graph-query extension compiled into the SQL frontend.                                                                                     | branch grust      |
| QueryGraph  | The semantic layer, Rust + Python: Croissant · CDIF · DID · ODRL projections, OSI business semantics, RBAC+ODRL dual gate, OpenLineage + attestations.                                    | 0.4.0 “Sentinel”  |

## GOSHAWK AND SENTINEL: THE INTEROPERABILITY RELEASES

**Real Ed25519 across languages** — Python signs, Rust verifies; Rust signs, Python verifies; same seed ⇒ same did:key; tampering fails on either side. **HTTP /v1 API** with TypeDID envelope auth (path- and body-bound; 401s carry receipts). **MCP servers in both languages** — Claude, LangChain, PydanticAI, LlamaIndex, CrewAI connect with zero adapters. **A2A agent card** published identically by both implementations. **Official OpenLineage conformance** — events schema-validated, deterministic UUIDv5 run ids shared across languages. Sentinel (0.4.0) adds the **governed navigator loop**: question → semantic search → policy receipts → SQL plans over allowed sources only → any LLM (or the deterministic baseline) → signed envelope + lineage + attestation.

## HELD EQUIVALENT BY TESTS

An executable cross-language contract: byte-identical navigator bundles, matching governance semantics in the multi-agent story, both crypto directions verified live, both CLIs’ lineage events schema-valid, identical agent cards, and a live auth round-trip — 49 Python + 40 Rust tests, CI on every push.

**Links:** [github.com/querygraph/{querygraph, qg-rust, qg-python, grust, typesec, lakecat}](https://github.com/querygraph/{querygraph, qg-rust, qg-python, grust, typesec, lakecat}) · [lakehq/sail](https://lakehq/sail) · books: [qg-rust/docs/book](https://qg-rust/docs/book) & [docs/guide](https://docs/guide) · [querygraph.ai](https://querygraph.ai)